

Decision Tree

Prof. Surya Prakash

Department of CSE, IIT Indore

Simple Health Dataset: Exercise & Diet vs. Heart Disease

Person	Exercise (hrs/week)	Diet Quality (1–10)	Heart Disease
1	0	3	Yes
2	1	4	Yes
3	2	5	Yes
4	3	6	No
5	4	7	No
6	5	8	No
7	3	4	Yes
8	4	6	No

Training Data

Test Person	Exercise (hrs/week)	Diet Quality (1–10)	Predicted Heart Disease
T1	2	5	
T2	4	7	

Test Data

Simple Health Dataset: Exercise & Diet vs. Heart Disease

Person	Exercise (hrs/week)	Diet Quality (1–10)	Heart Disease
1	0	3	Yes
2	1	4	Yes
3	2	5	Yes
4	3	6	No
5	4	7	No
6	5	8	No
7	3	4	Yes
8	4	6	No

Learning (from training samples):

if Exercise < 3
Heart Disease = Yes

else

if Diet < 6
Heart Disease = Yes

else

Heart Disease = No

Simple Health Dataset: Exercise & Diet vs. Heart Disease

Learning (from training samples):

```
if Exercise < 3
    Heart Disease = Yes
else
    if Diet < 6
        Heart Disease = Yes
    else
        Heart Disease = No
```

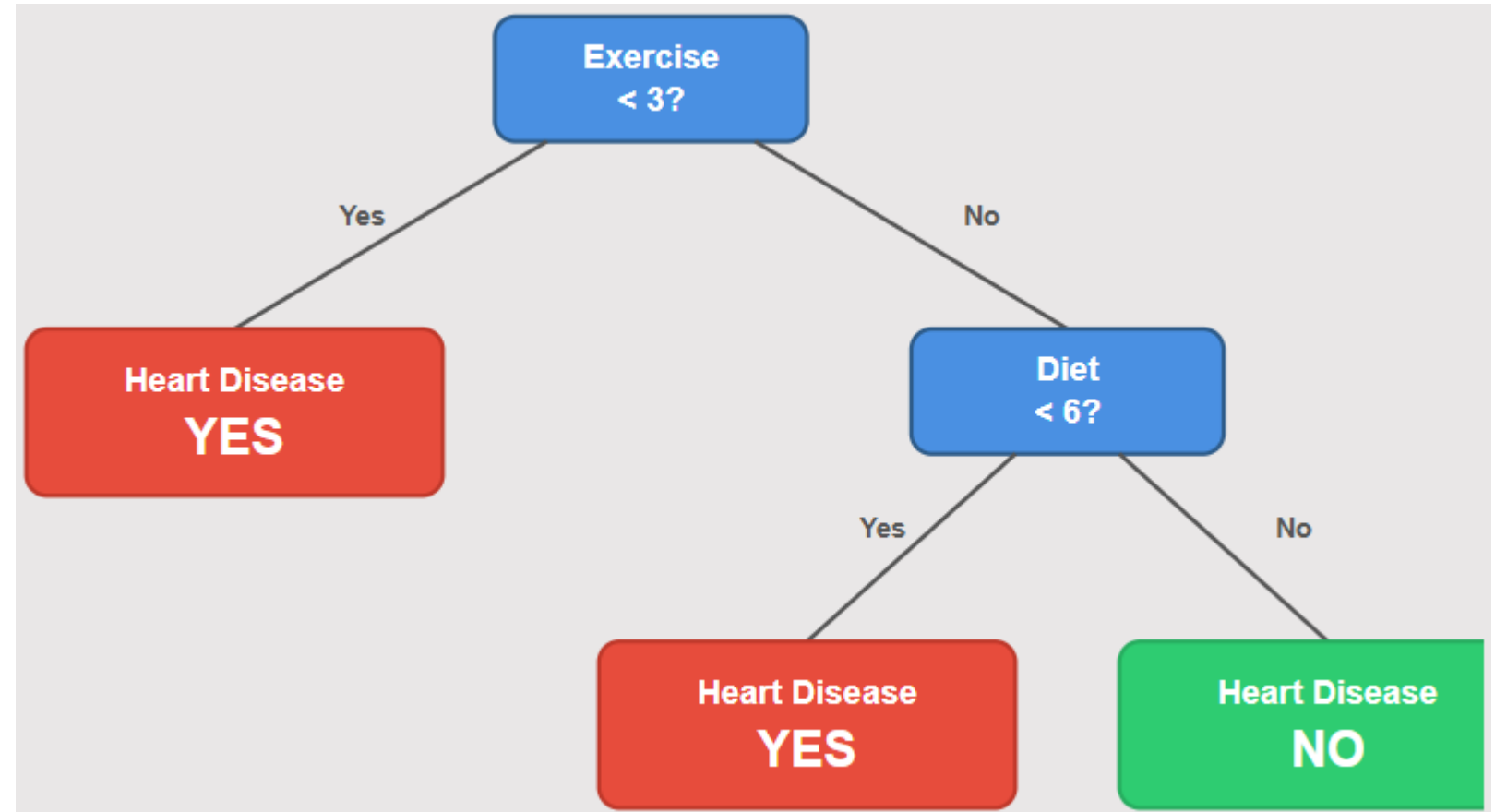
Test Person	Exercise (hrs/week)	Diet Quality (1–10)	Predicted Heart Disease
T1	2	5	Yes (low exercise)
T2	4	7	No (good exercise, good diet)

Test Data

Learning Represented in the form of a Tree

Learning (from training samples):

```
if Exercise < 3
    Heart Disease = Yes
else
    if Diet < 6
        Heart Disease = Yes
    else
        Heart Disease = No
```



Heart Disease Decision Tree

Another Observation

Person	Exercise (hrs/week)	Diet Quality (1–10)	Heart Disease
1	0	3	Yes
2	1	4	Yes
3	2	5	Yes
4	3	6	No
5	4	7	No
6	5	8	No
7	3	4	Yes
8	4	6	No

Learning (from training samples):

if Diet < 6

Heart Disease = Yes

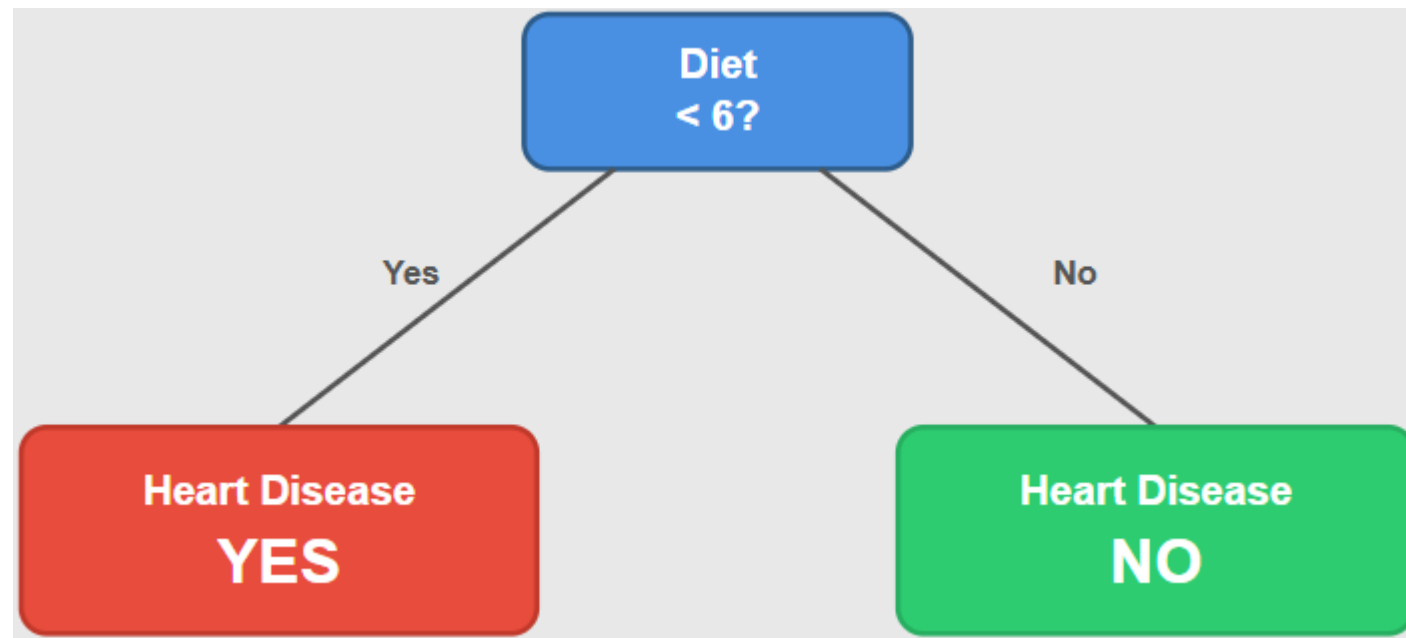
else

Heart Disease = No

Learning Represented in the form of a Tree

Learning (from training samples):

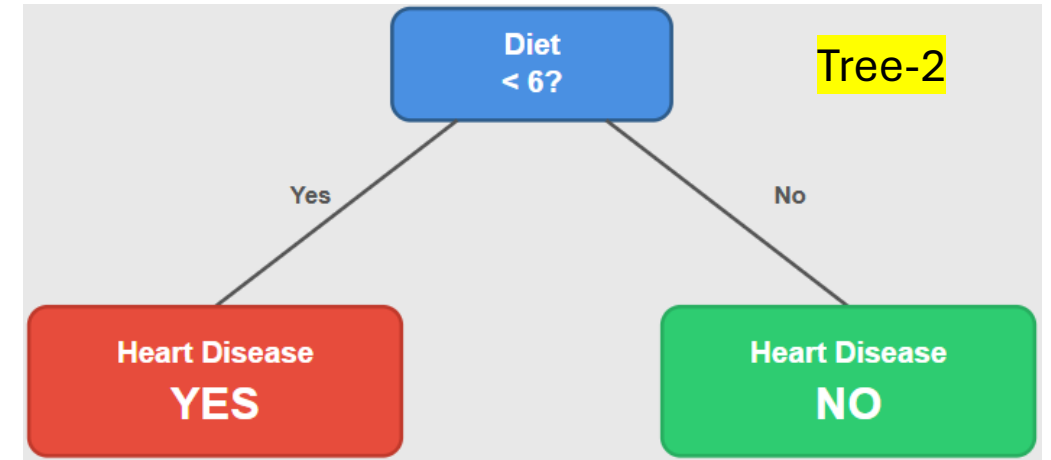
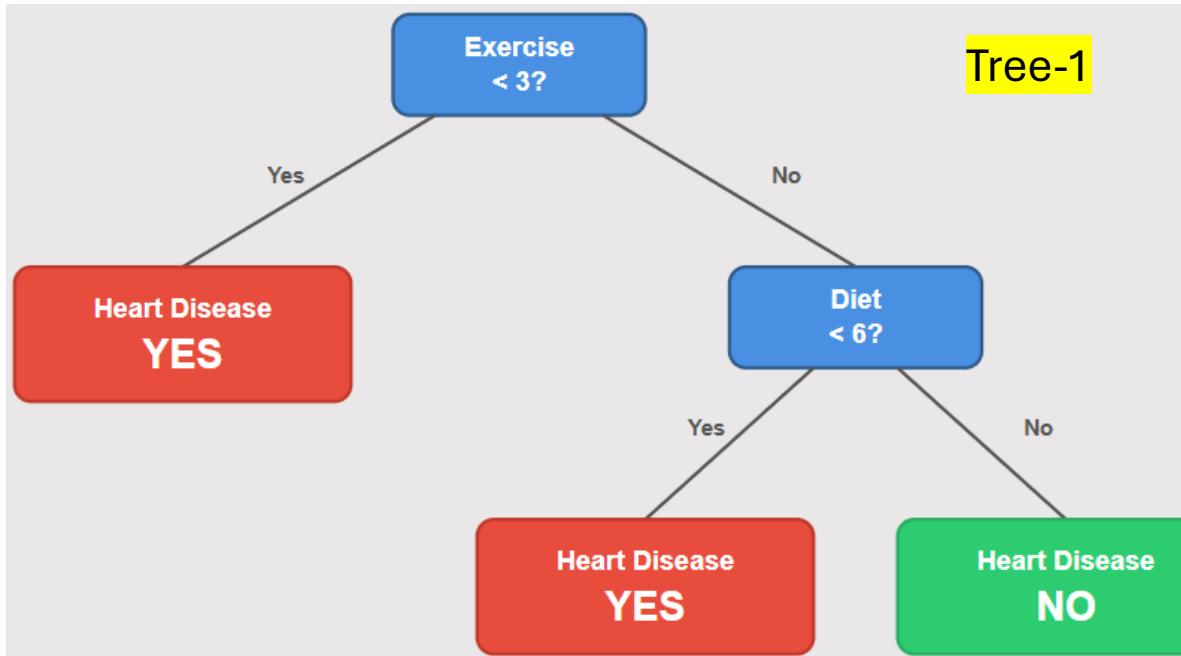
if Diet < 6
 Heart Disease = Yes
else
 Heart Disease = No



Heart Disease Decision Tree

Test Person	Exercise (hrs/week)	Diet Quality (1–10)	Predicted Heart Disease
T1	2	5	Yes (low exercise)
T2	4	7	No (good exercise, good diet)

Two Decision Trees – which one is better?



Heart Disease Decision Trees

- Both can perform classification, but which one is better?
- Tree-2 is better as it requires less inference time
- Decision tree building algorithm helps to get the best tree

Introduction to Decision Trees

- **What is a Decision Tree?**

- A decision tree is a flowchart-like structure used for decision making or classification.

- **Uses:**

- Classification
- Regression
- Feature selection

Structure of a Decision Tree

- **Root Node:**

- Represents the entire dataset and the **first decision point**

- **Internal Nodes:**

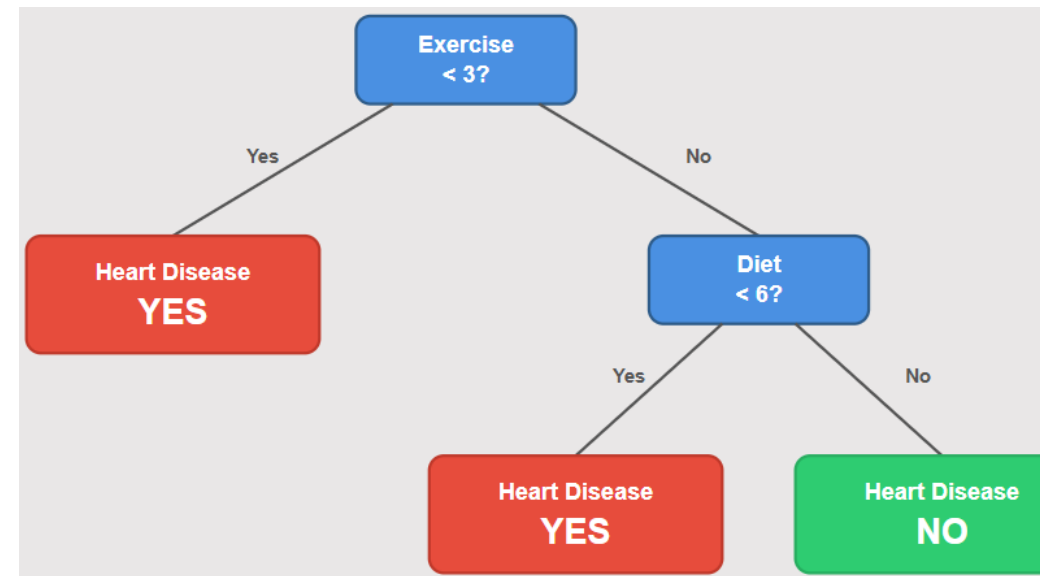
- Represent **decisions** based on attributes

- **Leaf Nodes:**

- Represent the **output class/label**

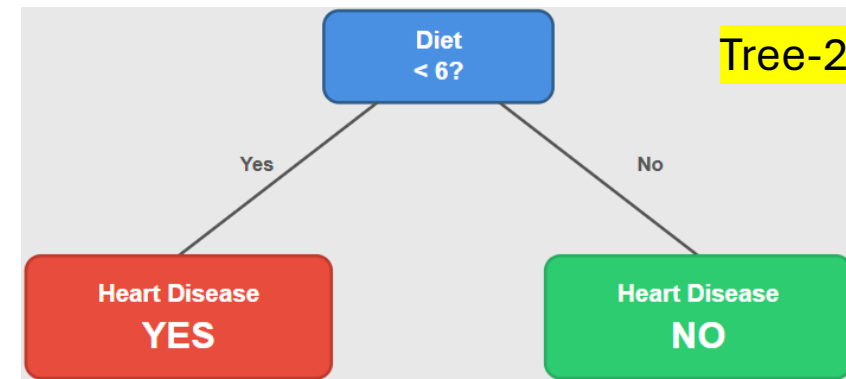
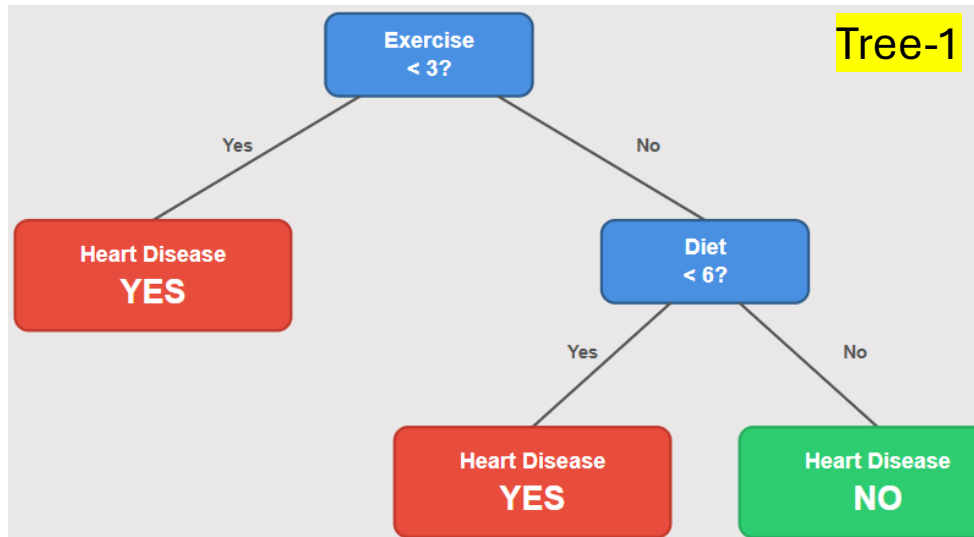
- **Edges:**

- Represent **outcomes** of the decision at a node



Building a Decision Tree

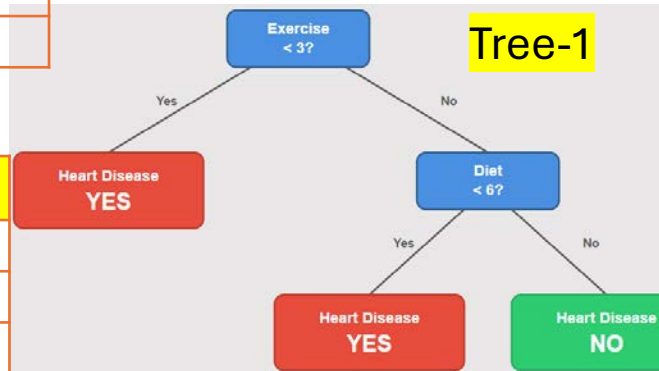
- The **main objective** in building a **Decision Tree** is to **select the best feature (attribute)** at each level (or node) to **split the data** so that the resulting subsets are as **pure** as possible.
- Purity of subsets?
 - This meaning the data points within each subset mostly belong to **one class**.



Building a Decision Tree

Person	Exercise (hrs/week)	Diet Quality (1-10)	Heart Disease
1	0	3	Yes
2	1	4	Yes
3	2	5	Yes
4	3	6	No
5	4	7	No
6	5	8	No
7	3	4	Yes
8	4	6	No

Person	Exercise (hrs/week)	Diet Quality (1-10)	Heart Disease
1	0	3	Yes
2	1	4	Yes
3	2	5	Yes



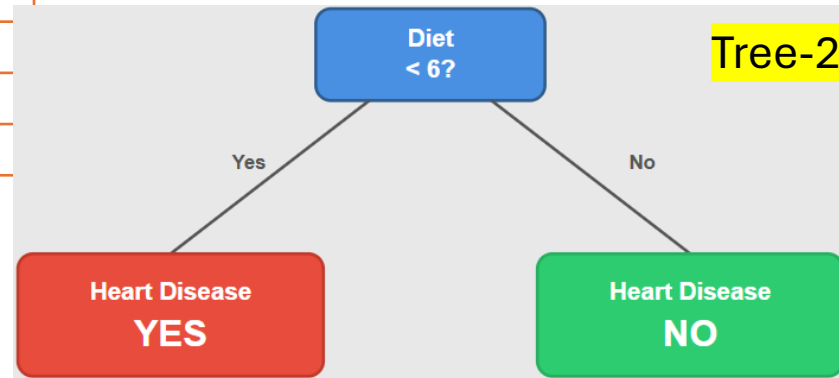
Person	Exercise (hrs/week)	Diet Quality (1-10)	Heart Disease
4	3	6	No
5	4	7	No
6	5	8	No
7	3	4	Yes
8	4	6	No

Person	Exercise (hrs/week)	Diet Quality (1-10)	Heart Disease
7	3	4	Yes

Person	Exercise (hrs/week)	Diet Quality (1-10)	Heart Disease
4	3	6	No
5	4	7	No
6	5	8	No
8	4	6	No

Building a Decision Tree

Person	Exercise (hrs/week)	Diet Quality (1-10)	Heart Disease
1	0	3	Yes
2	1	4	Yes
3	2	5	Yes
4	3	6	No
5	4	7	No
6	5	8	No
7	3	4	Yes
8	4	6	No



Person	Exercise (hrs/week)	Diet Quality (1-10)	Heart Disease
1	0	3	Yes
2	1	4	Yes
3	2	5	Yes
7	3	4	Yes

Person	Exercise (hrs/week)	Diet Quality (1-10)	Heart Disease
4	3	6	No
5	4	7	No
6	5	8	No
8	4	6	No

Building a Decision Tree

- **ID3: Iterative Dichotomiser 3:**
 - ID3 algorithm is specifically designed for **classification**, not regression.
 - The earlier versions (ID1 and ID2) were unpublished or internal prototypes.
 - ID3 was the first version to be formally published and widely adopted.

ID3 Algorithm

- Uses the concept of
 - Entropy
 - Information gain

ID3 Algorithm

- **Entropy:**

- In the context of **information theory and decision trees**, **entropy** is a measure of **impurity or uncertainty** in a dataset.

If a dataset S has c classes, and the probability of class i is p_i , then the **entropy** of S is:

$$Entropy(S) = - \sum_{i=1}^c p_i \log_2(p_i)$$

- **Intuition**

- If all examples belong to a **single class** → entropy = **0** (pure, no uncertainty).
- If examples are **evenly split** among classes → entropy is **maximum** (maximum uncertainty).

Consider Dataset - 14 samples

Outlook	Temperature	Humidity	Windy	Play
sunny	hot	high	false	no
sunny	hot	high	true	no
overcast	hot	high	false	yes
rainy	mild	high	false	yes
rainy	cool	normal	false	yes
rainy	cool	normal	true	no
overcast	cool	normal	true	yes
sunny	mild	high	false	no
sunny	cool	normal	false	yes
rainy	mild	normal	false	yes
sunny	mild	normal	true	yes
overcast	mild	high	true	yes
overcast	hot	normal	false	yes
rainy	mild	high	true	no

Example

Suppose we have a dataset of 14 instances of "Play Tennis":

- 9 "Yes"
- 5 "No"

So:

$$p(Yes) = \frac{9}{14}, \quad p(No) = \frac{5}{14}$$

$$Entropy(S) = - \left(\frac{9}{14} \log_2 \frac{9}{14} + \frac{5}{14} \log_2 \frac{5}{14} \right) \approx 0.94$$

Information Gain

- **Information Gain** measures how much **uncertainty (entropy)** is **reduced** when we split a dataset on an attribute.
- Mathematically:

$$IG(S, A) = Entropy(S) - \sum_{v \in Values(A)} \frac{|S_v|}{|S|} Entropy(S_v)$$

Where:

- S = original dataset
- A = attribute we split on
- $Values(A)$ = all possible values of attribute A
- S_v = subset of S where $A = v$
- $\frac{|S_v|}{|S|}$ = weight (proportion of samples)

Information Gain

- **Intuition**

- **High IG** → attribute gives a “clearer split” (reduces randomness a lot).
- **Low IG** → attribute doesn’t help much in classification.

- Decision Trees (like ID3) choose the **attribute with maximum IG** at each node.

ID3 Algorithm

- **Steps:**

- Calculate Entropy of the dataset
- Compute Information Gain for each attribute
- Select attribute with highest Information Gain as the decision node
- Repeat recursively for each child node until stopping criteria is met

Entropy and Information Gain (Simple View)

- Entropy (S)

$$\text{Entropy}(S) = -p_{\text{yes}} \log_2(p_{\text{yes}}) - p_{\text{no}} \log_2(p_{\text{no}})$$

- Information Gain

$$\text{Gain}(S, \text{Attribute}) = \text{Entropy}(S) - \text{Weighted Entropy of Children}$$

- Use Entropy to measure impurity.
 - After splitting, we want node to become pure
- Choose attribute with highest gain for the split.

Decision Tree Example : Dataset - 14 samples

Outlook	Temperature	Humidity	Windy	Play
sunny	hot	high	false	no
sunny	hot	high	true	no
overcast	hot	high	false	yes
rainy	mild	high	false	yes
rainy	cool	normal	false	yes
rainy	cool	normal	true	no
overcast	cool	normal	true	yes
sunny	mild	high	false	no
sunny	cool	normal	false	yes
rainy	mild	normal	false	yes
sunny	mild	normal	true	yes
overcast	mild	high	true	yes
overcast	hot	normal	false	yes
rainy	mild	high	true	no

Final Decision Tree

Outlook?

├ Sunny

└ Humidity?

└ High → No

└ Normal → Yes

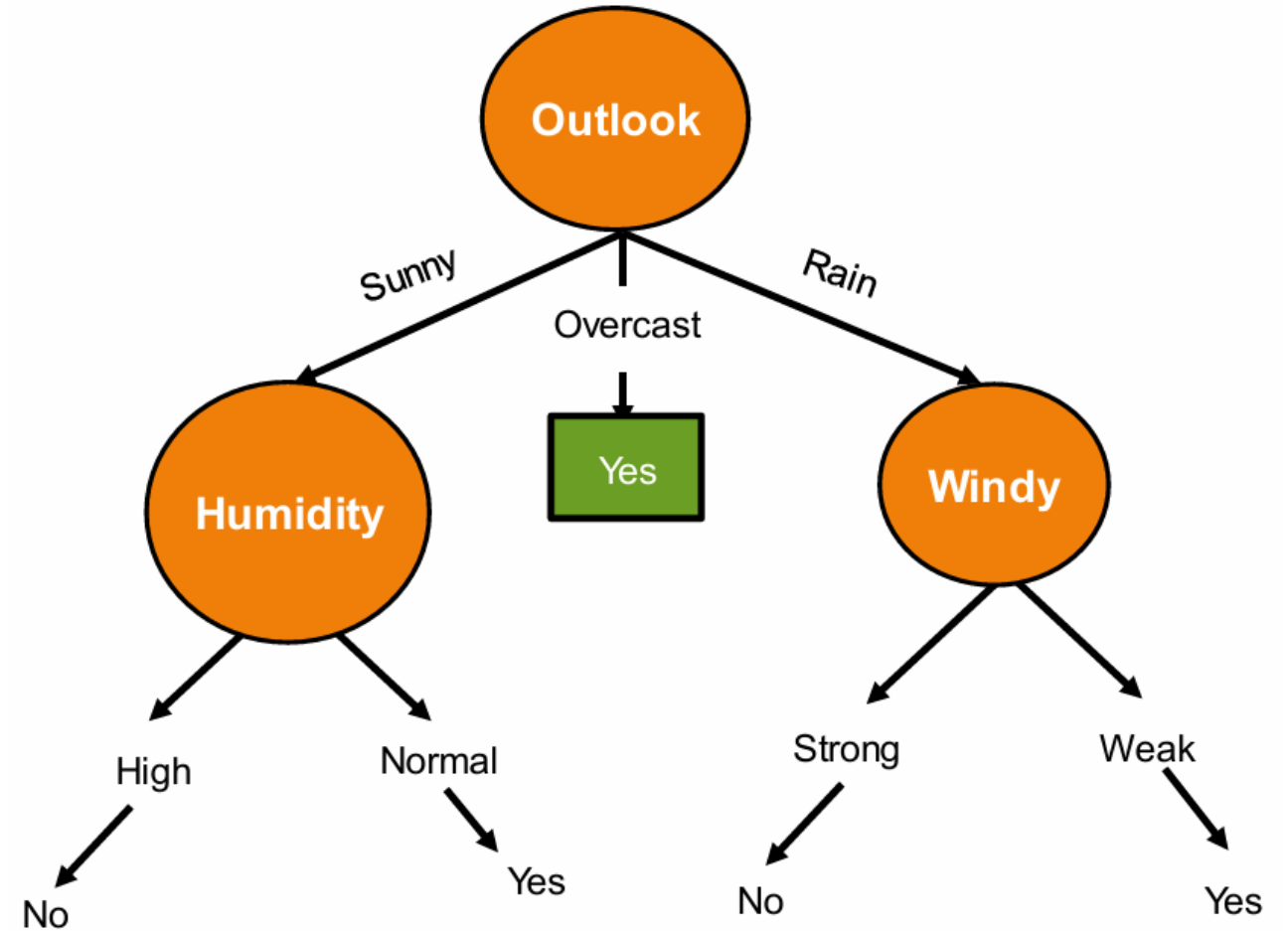
├ Overcast → Yes

└ Rainy

└ Windy?

└ False → Yes

└ True → No



Decision Tree Classifier - Code

1. Import required libraries

```
import pandas as pd
from sklearn.tree import DecisionTreeClassifier
from sklearn.preprocessing import LabelEncoder
```

2. Create the Play Tennis dataset

```
data =
{
    'Outlook':
    ['Sunny', 'Sunny', 'Overcast', 'Rain', 'Rain', 'Rain', 'Overcast', 'Sunny', 'Sunny', 'Rain', 'Sunny', 'Overcast', 'Overcast', 'Rain'],
    'Temperature':
    ['Hot', 'Hot', 'Hot', 'Mild', 'Cool', 'Cool', 'Cool', 'Mild', 'Cool', 'Mild', 'Mild', 'Mild', 'Hot', 'Mild'],
    'Humidity':
    ['High', 'High', 'High', 'High', 'Normal', 'Normal', 'Normal', 'High', 'Normal', 'Normal', 'Normal', 'Normal', 'High', 'Normal', 'High'],
    'Wind':
    ['Weak', 'Strong', 'Weak', 'Weak', 'Weak', 'Strong', 'Strong', 'Weak', 'Weak', 'Weak', 'Strong', 'Strong', 'Weak', 'Strong'],
    'Play': ['No', 'No', 'Yes', 'Yes', 'Yes', 'No', 'Yes', 'No', 'Yes', 'Yes', 'Yes', 'Yes', 'Yes', 'Yes', 'No']
}
```

Convert dictionary to DataFrame

```
df = pd.DataFrame(data)
```

Decision Tree Classifier - Code

3. Encode categorical features into numeric values

```
le = LabelEncoder()  
for column in df.columns:  
    df[column] = le.fit_transform(df[column])
```

4. Separate features (X) and target (y)

```
X = df.drop('Play', axis=1)    # Input features  
y = df['Play']                # Target variable
```

5. Create and train the Decision Tree model

```
model = DecisionTreeClassifier(criterion='entropy') # Using Information Gain  
model.fit(X, y)
```

6. Make a prediction

Example input:

```
# Outlook = Sunny, Temperature = Cool, Humidity = High, Wind = Strong  
sample = [[2, 0, 0, 1]] # Encoded values (may vary based on encoding)  
prediction = model.predict(sample)
```

7. Print result

```
print("Prediction:", "Yes" if prediction[0] == 1 else "No")
```

End